

Do It Yourself Privacy Guard (DIYPG)

Proposé par Vincent Dugat



Table des matières

1 Introduction	2
2 Le système RSA.	2
2.1 Principe général	2
2.2 Fonctionnement pratique	3
2.2.1 Création des clés	3
2.2.2 Chiffrement du message	3
2.2.3 Déchiffrement du message	4
2.2.4 Exemple	4
2.2.5 Attaque pour casser le chiffre	4
3 Le sujet	4
3.1 Phase 1 : code Moodle à lire et comprendre et code à écrire.	4
3.1.1 Code fourni, et disponible sur la page Moodle	4
3.1.2 IMPLEMENTATION : cahier des charges de la phase 1.0.	5
3.1.3 IMPLEMENTATION : cahier des charges de la phase 1.1.	6
3.1.4 IMPLEMENTATION : cahier des charges de la phase 1.2.	6
3.1.5 IMPLEMENTATION : cahier des charges de la phase 1.3 : outils pour le chiffrement : conversion base 64	6
3.2 Phase 2 : rendre le chiffrement plus résistant : utilisation de blocs de quatre octets et de GMP	6
3.2.1 IMPLEMENTATION : cahier des charges de la phase 2	7
3.3 PHASE 3 : interprète de commande et annuaire de clefs	7
3.3.1 IMPLEMENTATION : cahier des charges de la phase 3	8
3.4 PHASE 4 : signatures et hash	8
3.4.1 IMPLEMENTATION : cahier des charges de la phase 4	8
3.5 PHASE 5 : Registre d'authentification des clefs publiques	8

3.5.1	Cahier des charges d'implémentation	9
3.5.2	Fonctionnalités supplémentaires	10

1 Introduction

Le projet concerne la cryptographie asymétrique selon le système RSA (simplifié). Dans ce système il y a deux clés : une clé publique servant à envoyer un message chiffré à quelqu'un, et une clé privée servant à ce quelqu'un à déchiffrer le message. Le même système peut servir aussi à signer électroniquement un message avec une clé privée et à vérifier une signature avec la clé publique.

Le sujet sera découpé en plusieurs parties. Chaque partie sera constituée d'une explication et d'un cahier des charges de réalisation. Il est conseillé de faire une première lecture globale de tout le sujet pour comprendre où le sujet cherche à vous amener.

Chaque partie fera l'objet d'un dépôt de code et d'une recette en présentiel, c'est à dire d'une présentation du fonctionnement du programme. Le code déposé devra être raisonnablement commenté, accompagné d'un fichier makefile pour la compilation et d'un README.

!! Spoiler !! : il s'agit de programmer un système du type Gnu Privacy Guard (GPG).

2 Le système RSA.

Le sujet est basé sur le document suivant.

<https://zestedesavoir.com/tutoriels/723/la-cryptographie-asymetrique-avec-rsa/>

C'est là que vous trouverez les détails des explications.

Le système cryptographique RSA a été inventé en 1977, et publié en 1978, par trois mathématiciens qui travaillaient alors au MIT : Ronald Rivest, Adi Shamir et Leonard Adleman. Comme vous pouvez le constater, le nom de l'algorithme est simplement tiré des initiales de leurs noms de famille. La grande réussite de leur travail en commun est sans doute due au fait que leurs compétences se complètent.¹

Le système RSA est différent des systèmes cryptographiques dits symétriques où toute la solidité du chiffrement repose sur une « clé » secrète, qui doit être connue à la fois de l'envoyeur et du récipiendaire.

Le message d'origine, avant toute forme de chiffrement, est appelé message en clair. Une fois qu'un algorithme de chiffrement lui a été appliqué, on l'appelle message chiffré.

Un algorithme de chiffrement/déchiffrement décrit une suite d'opérations qui est la même pour tout le monde. Or, si tout le monde utilise exactement la même méthode pour protéger ses données, tout le monde peut lire les messages des autres, puisque l'algorithme de déchiffrement est le même que pour ses propres messages. C'est pourquoi les algorithmes laissent toujours un tout petit espace pour que l'utilisateur puisse les personnaliser : en gardant secret cet élément de personnalisation, que l'on appelle une (ou des) clé(s), la protection est assurée. Il s'agit généralement d'un mot ou d'une phrase de passe.

Pour désigner les personnes impliquées dans une opération de chiffrement, les spécialistes ont pris l'habitude de nommer Alice et Bob, les agents qui tentent de s'écrire sans être dérangés par l'espion Eve, qui leur veut du mal sans qu'on sache trop pourquoi.

2.1 Principe général

La cryptographie asymétrique, ou cryptographie à clef publique est fondée sur l'existence des fonctions à sens unique et à brèche secrète. Les fonctions à sens unique sont des fonctions mathématiques telles qu'une fois appliquées à un message, il est extrêmement difficile de retrouver le message original.

1. Rivest est un grand spécialiste de la cryptographie, la discipline qui cherche à protéger les messages d'une lecture inopportune : il est notamment à l'origine de l'algorithme MD5 ; Shamir est au contraire un spécialiste de la cryptanalyse, dont le but est de casser les protections mises en place par la cryptographie : il est notamment connu pour avoir cassé en 1982 le cryptosystème de Merkle-Hellman, un algorithme proche de RSA mais reposant sur une base mathématique un peu différente ; Adleman, quant à lui, travaillait à l'époque sur la bio-informatique, en gros, des ordinateurs utilisant de l'ADN pour faire leurs calculs.

L'existence d'une brèche secrète permet cependant à la personne qui a conçu la fonction à sens unique de décoder facilement le message grâce à un élément d'information qu'elle possède, appelé clef privée.

Supposons qu'Alice souhaite recevoir un message secret de Bob sur un canal susceptible d'être écouté par l'attaquant passif Eve :

- Alice transmet à Bob une fonction à sens unique pour laquelle elle seule connaît la brèche secrète
- Bob utilise la fonction transmise par Alice pour chiffrer son message secret
- Alice réceptionne le message chiffré puis le décode grâce à la brèche secrète
- Si Eve réceptionne également le message alors qu'il circule sur le canal public, elle ne peut le décoder, même si elle a également intercepté l'envoi de la fonction à sens unique, car elle n'a pas connaissance de la brèche secrète

La terminologie classiquement retenue est :

- pour la fonction à sens unique : «clef publique»
- pour la brèche secrète : «clef privée»

En pratique, sont utilisées des fonctions de chiffrement classiques, les termes «clef publique» et «clef privée» correspondant alors à des paramètres employés pour ces fonctions.

Pour des explications plus détaillées il est conseillé de lire le document de référence :

<https://zestedesavoir.com/tutoriels/723/la-cryptographie-asymetrique-avec-rsa/>

Vous pouvez aussi lire :

https://fr.wikipedia.org/wiki/Cryptographie_asym%C3%A9trique

https://fr.wikipedia.org/wiki/Chiffrement_RSA

2.2 Fonctionnement pratique

Je ne vais pas vous mentir : c'est des maths.

2.2.1 Création des clés

L'étape de création des clés est à la charge d'Alice. Elle n'intervient pas à chaque chiffrement car les clés peuvent être réutilisées. La difficulté première, que ne règle pas le chiffrement, est que Bob soit bien certain que la clé publique qu'il détient est celle d'Alice. Le renouvellement des clés n'intervient que si la clé privée est compromise, ou par précaution au bout d'un certain temps (qui peut se compter en années).

1. Choisir p et q , deux nombres premiers distincts ;
2. calculer leur produit $n = p.q$, appelé module de chiffrement ;
3. calculer $\phi(n) = (p-1)(q-1)$ (c'est la valeur de l'indicatrice d'Euler en n) ;
4. choisir un entier naturel e premier avec $\phi(n)$ et strictement inférieur à $\phi(n)$, appelé exposant de chiffrement ;
5. calculer l'entier naturel d , inverse de e modulo $\phi(n)$, et strictement inférieur à $\phi(n)$, appelé exposant de déchiffrement ; d peut se calculer efficacement par l'algorithme d'Euclide étendu.

Comme e est premier avec $\phi(n)$, d'après le théorème de Bachet-Bézout il existe deux entiers d et k tels que $e.d = 1 + k.\phi(n)$, c'est-à-dire que $ed \equiv 1 \pmod{\phi(n)}$: e est bien inversible modulo $\phi(n)$.

Le couple (n, e) , ou (e, n) , est la clé publique du chiffrement, alors que sa clé privée est le nombre d , sachant que l'opération de déchiffrement ne demande que la clef privée d et l'entier n , connu par la clé publique

2.2.2 Chiffrement du message

Si M est un entier naturel strictement inférieur à n représentant un message, alors le message chiffré sera représenté par :

$C \equiv M^e \pmod{n}$, l'entier naturel C étant choisi strictement inférieur à n .

2.2.3 Déchiffrement du message

Pour déchiffrer C , on utilise d , l'inverse de e modulo $(p-1)(q-1)$, et l'on retrouve le message clair M par :

$$M \equiv C^d \pmod{n}.$$

2.2.4 Exemple

Un exemple avec de petits nombres premiers (en pratique il faut de très grands nombres premiers) :

1. on choisit deux nombres premiers $p = 3, q = 11$;
2. leur produit $n = 311 = 33$ est le module de chiffrement ;
3. $\phi(n) = (3-1)(11-1) = 210 = 20$;
4. on choisit $e = 3$ (premier avec 20) comme exposant de chiffrement ;
5. l'exposant de déchiffrement est $d = 7$, l'inverse de 3 modulo 20 (en effet $e.d = 3 \times 7 = 1 \pmod{20}$).

La clé publique d'Alice est $(n, e) = (33, 3)$, et sa clé privée est $(n, d) = (33, 7)$.

Bob transmet alors un message à Alice.

- Chiffrement de $M = 4$ par Bob avec la clé publique d'Alice : $4^3 \pmod{33}$, le chiffré est $C = 31$ que Bob transmet à Alice ;
- Déchiffrement de $C = 31$ par Alice avec sa clé privée : $31^7 \pmod{33}$, Alice retrouve le message initial $M = 4$.

2.2.5 Attaque pour casser le chiffre

Il faut distinguer les attaques par la force brute, qui consistent à retrouver p et q sur base de la connaissance de n uniquement, et les attaques sur base de la connaissance de n mais aussi de la manière dont p et q ont été générés, du logiciel de cryptographie utilisé, d'un ou plusieurs messages éventuellement interceptés etc.

La sécurité de l'algorithme RSA contre les attaques par la force brute repose sur deux conjectures :

1. « casser » RSA de cette manière nécessite la factorisation du nombre n en le produit initial des nombres p et q ,
2. avec les algorithmes classiques, le temps que prend cette factorisation croît exponentiellement avec la longueur de la clé.

3 Le sujet

3.1 Phase 1 : code Moodle à lire et comprendre et code à écrire.

La phase 1 est divisée en quatre parties. On commence par la phase 1.0 qui implémente les outils de nombres premiers, factorisation, algorithme d'Euclide étendu (Bézout) ; et la création des clefs. On se limite dans cette phase à des `uint64`, c'est à dire des entiers non signés codés sur 64 bits. Les nombres premiers générés et les calculs sont limités en taille, attention au dépassement de capacité, il n'est pas géré en C.

3.1.1 Code fourni, et disponible sur la page Moodle

- `rsa_common_header.h` contient les types utilisés dans `rsa_tools.c`. Vous pouvez tout changer mais il faudra adapter les fonctions de `rsa_tools.c`
- `rsa_tools.c` fonctions de génération de nombres aléatoires, tests de primalité, génération aléatoire de nombres premiers et de clefs.
- `other_base64.c` fonctions de conversion en base 64 et vice-versa.
- `Bézout.c`
- `Sha-256` fonctions de hash pour les digests

Un peu plus de détails sur le fichier : rsa_tools.c Il utilise la fonction bezoutRSA du fichier b  zout.c, lui aussi disponible sur Moodle.

1. int premier (uint n) : test de primarit   avec le crible d'Erathost  ne, renvoie 1 si le nombre est premier, 0 sinon
 2. uint random_uint(uint min,uint max) : g  n  re un uint al  atoire entre min et max. min et max sont des uint, renvoie $n : \min \leq n \leq \max$
 3. int decompose (uint facteur[], uint64 n) : d  compose un entier en facteurs premiers facteur.(brute force), param[in] : n l'entier, param[out] : facteur = tableau de facteurs, returns la dimension du tableau
 4. uint puissance(uint a, uint e) : puissance russe, param[in] : a l'entier et e l'exposant, returns : a^e
 5. uint64 puissance_mod_n (uint64 a, uint64 e, uint64 n) : puissance modulaire, calcule $a^e \bmod n$, ATTENTION : $a \times a$ peut d  passer la capacit   d'un uint64
- Se r  f  rer    :

// <https://www.labri.fr/perso/betrema/deug/poly/exp-rapide.html>

6. uint genereUint(uint max,int *cpt) : g  n  re un nombre premier inf  rieur    max. param[in] max : la borne sup, param[out] cpt : nombre d'essais, returns : le nombre premier. cpt compte le nombre d'essai pour trouver le nombre premier.
7. int rabin (uint a, uint n) : test de Rabin sur la pimarit   d'un entier, c'est un test statistique
8. uint genereUintRabin(uint max,int *cpt) : fournit un nombre premier v  rifi   avec le test de rabin
9. uint pgcdFast(uint a,uint b) : pgcd rapide de a et b, param[in] a et b les deux entiers, pgcd(a,b).
10. void genKeysRabin(rsaKey_t *pubKey,rsaKey_t *privKey) : g  n  re une paire de clefs, param[out] pubKey : clef publique, param[out] privKey : clef priv  e
11. void inputKey(uint64 E,uint64 N,rsaKey_t *key) : pour debug : permet de saisir une clef
12. long bezoutRSA(uint a,uint b,long *u,long *v) : r  cup  re (r,u,v) de B  zout. Si u est n  gatif on le remplace par le premier qui est sup  rieur    2.
13. long bezout(uint a,uint b,long *u,long *v) : impl  mente l'algorithme d'Euclide   tendu

https://fr.wikipedia.org/wiki/Algorithme_d%27Euclide_%C3%A9tendu

Fonction utilis  e par la pr  c  dente. param[in] : a, b entiers (naturels), param[out] : u, v entiers relatifs tels que $r = \text{pgcd}(a, b)$ et $r = a * u + b * v$, returns r entier

3.1.2 IMPLEMENTATION : cahier des charges de la phase 1.0.

1. Comprendre le code et savoir utiliser les fonctions.
2. Ecrire une ou des fonction(s) affichant les clefs au format hexad  cimal.
3. Faire un main de test    inclure dans le livrable de cette partie pour g  n  rer et afficher une paire de clefs.
4. Faire une fonction pour sauver une paire de clefs dans un fichier et pour la charger dans le programme.
5. Chiffrer et d  chiffrer un caract  re UTF8    titre de test. Faire une boucle r  p  tant ce test avec diff  rentes paires de clefs.
6. Modifiez la valeur de la constante MAXPRIME pour cerner le d  passement de capacit   des uint64. Dans le header elle est fix  e    10000. Ces tests seront   galement    inclure dans les livrables.
7. Ecrire dans un README un bilan de cette partie : valeur de MAXPRIME, exhaustivit   des tests.

3.1.3 IMPLEMENTATION : cahier des charges de la phase 1.1.

Dans cette phase, on va chiffrer et déchiffrer un message caractère par caractère écrit dans un tableau. Cependant **c'est ce qu'il ne faut pas faire car cette méthode est sensible à l'attaque statistique**. On prend chaque octet du code utf8 que l'on considère comme un entier pour faire les calculs.

1. Implémentez une fonction qui chiffre un tableau d'octets (uint8), le résultat sera un tableau d'octets (uint8)
2. Implémentez une fonction qui déchiffre un tableau d'octets (uint8), le résultat sera un tableau d'octets (uint8)
3. Définissez une politique de tests pour ces fonctions.

3.1.4 IMPLEMENTATION : cahier des charges de la phase 1.2.

On ajoute à la phase précédente la gestion des fichiers. Cette fois le message à chiffrer ou déchiffrer sera lu dans un fichier et le résultat sera écrit dans un fichier.

1. Implémentez une fonction qui chiffre un message lu dans un fichier, le résultat sera écrit dans un fichier. La clef de chiffrement est passée en paramètre.
2. Implémentez une fonction qui déchiffre un message lu dans un fichier, le résultat sera écrit dans un fichier. La clef de déchiffrement est passée en paramètre.
3. Ecrire un main de test pour gérer l'appel des fonctions.
4. Définir une politique de tests pour valider ces fonctions et les implémentations des algorithmes de chiffrement.

3.1.5 IMPLEMENTATION : cahier des charges de la phase 1.3 : outils pour le chiffrement : conversion base 64

Le processus de codage base64 consiste à coder chaque groupe de 24 bits successifs de données d'un fichier binaire par une chaîne de 4 caractères. On procède de gauche à droite, en concaténant 3 octets pour créer un seul groupement de 24 bits (8 bits par octet). Ils sont alors séparés en 4 nombres de seulement 6 bits (qui en binaire ne permettent que 64 combinaisons). Chacune des 4 valeurs est enfin représentée (codée) par un caractère de l'alphabet retenu. Ainsi 3 octets quelconques sont remplacés par 4 caractères, choisis pour être compatibles avec tous les systèmes existants.

Cette conversion est utile pour écrire des données binaires dans des fichiers et, par exemple, les transférer par mail.

- Ecrire une fonction qui convertit un tableau d'octet (uint8) en tableau de caractères en base64
- Ecrire une fonction qui convertit un tableau base64 en tableau de caractères binaire
- Ecrire les tests convertissant des entiers uint32, uint64, des tableaux de caractères, des clefs en base64 et vice-versa.

Commande Unix pour vérifier la transcription si besoin :

```
$ echo -n 'Hi!' | base64  
SGkh
```

- Implémenter une fonction de conversion d'un fichier binaire vers un fichier en base64.
- Implémenter une fonction qui fait l'opération inverse de la précédente.
- Faire un bilan des tests de validation.

3.2 Phase 2 : rendre le chiffage plus résistant : utilisation de blocs de quatre octets et de GMP

Pour brouiller les cartes et gêner l'analyse statistique on peut prendre un bloc de plusieurs octets dans le message, en faire un entier et l'utiliser pour les calculs. On prend des blocs de quatre octets ce qui correspond à la taille d'un uint32 (unsigned int). C'est une simplification. Les systèmes cryptographiques effectivement employés utilisent des blocs bien plus grands.

Le problème est qu'avec ce système les calculs n'ont plus aucune chance de tenir sur un `uint64`. Donc on va utiliser une bibliothèque de calcul en précision illimitée : GMP. GMP est un code libre. Sa syntaxe est un peu lourde aussi allons-nous garder les clefs sur 64 bits (`uint64`). Nous pourrions cette fois nous approcher plus près de la limite `UINT64_MAX` de `stdint.h`

GMP La bibliothèque GMP (GNU Multiple Precision) est disponible ici :

<https://gmplib.org/>

C'est une bibliothèque logicielle de calcul multiprécision sur des nombres entiers, rationnels et en virgule flottante. Voici un tuto d'installation :

<https://openclassrooms.com/fr/courses/1362801-la-cryptographie-asymetrique-rsa/1363119-gmp-arithmetic-without-limitation>

Nous allons devoir ré-écrire certains des outils de théorie des nombres avec les fonctions de cette bibliothèque. GMP contient déjà en standard des fonctions qui vont nous servir comme `mpz_powm` qui fait la puissance modulaire.

3.2.1 IMPLEMENTATION : cahier des charges de la phase 2

On peut maintenant envisager de reprendre le calcul de chiffrement par blocs.

1. `uint convert_4byte2int(uchar *b)` : conversion de 4 octets en un int selon l'ordre little endian **Ce code est donné sur Moodle.**
2. `void convertInt2uchar(uint nb, uchar *tab4bytes)` : convert from an unsigned int to a 4-byte array **Ce code est donné sur Moodle.**
3. Ecrire la fonction `void puissance_mod_n_gmp(mpz_t res, uint64 a, uint64 e, uint64 n)` qui adapte le calcul de puissance modulaire $a^e \bmod n$ à GMP.
4. Ecrire les fonctions utiles pour :
 - (a) chiffrer un bloc et déchiffrer un bloc chiffré,
 - (b) chiffrer un message par blocs depuis un fichier et écrire le chiffré dans un fichier,
 - (c) déchiffrer un message chiffré par blocs, depuis un fichier et écrire le résultat dans un fichier.
 - (d) définir une méthode pour envoyer par mail les clefs publiques à des correspondants : base 64, conversion en hexadécimal utf8, ou autre, faites un choix. Ce choix peut impliquer des fonctions supplémentaires de conversion. Echanger vos clefs publiques dans l'équipe.

Pour faire transiter les messages chiffrés sur un réseau sans que certains octets soient interprétés comme des caractères de contrôle il nous faut aussi :

1. Une fonction qui convertit le chiffré en base64 et met le résultat dans un nouveau fichier.
2. Une fonction qui convertit un fichier en base64 en binaire et met le résultat dans un nouveau fichier.
3. Définir des tests de validation pour valider les fonctions écrites.

Vous pouvez à ce stade envoyer des pièces jointes chiffrées dans des courriels en utilisant la clef publique de vos correspondants. C'est une bonne validation du code. Le mieux est d'échanger des messages avec une autre équipe qui utilisera son code.

3.3 PHASE 3 : interprète de commande et annuaire de clefs

Pour gérer les clefs, celles de l'utilisateur et de ses contacts, on va créer un annuaire sous forme d'un interprète en ligne de commande permettant de :

1. générer des paires de clés et leur associer un identificateur (table des symboles)
2. chiffrer, déchiffrer avec les clés en utilisant l'identificateur associé
3. supprimer des clés

4. lister toutes les clefs
5. convertir un fichier binaire en base64
6. convertir un fichier base64 en binaire
7. sauvegarde des clés dans un fichier. Ce fichier devra être protégé en lecture et modification (chiffré avec mot de passe ou des clefs RSA ou droits système, etc.)

Des spécifications plus précises sont données dans le document `interprète.pdf` sur Moodle

3.3.1 IMPLEMENTATION : cahier des charges de la phase 3

Implémenter l'interprète et les fonctionnalités demandées. Vous devez aussi à ce stade faire un *main* qui appellera l'interprète. Ce sera le programme principal de l'application.

3.4 PHASE 4 : signatures et hash

La signature d'un document utilise aussi les clefs cryptographiques. Si le chiffage est destiné à assurer la confidentialité des documents (personne d'autre que la personne ayant la clef privée ne peut les lire), la signature concerne l'intégrité (le document n'a pas été modifié) et la paternité (le document a bien été envoyé par la personne qui le prétend).

Le document peut être transmis en clair, s'il est publique, ou lui-même chiffré. Pour signer un document on va calculer son hash à l'aide d'une fonction de hashage (ici sha 256). Le résultat s'appelle le "digest". Le digest va alors être chiffré par la clef privée de l'expéditeur. Il transmet alors le document et la signature.

Pour vérifier la signature, on utilise la clef publique de l'expéditeur pour retrouver le hashé. On hash le document et on vérifie que les deux hashés sont bien identiques.

Avec un document chiffré on agit de la même manière.

Pour des raisons de sécurité, Il est fortement déconseiller d'utiliser la même paire de clefs pour chiffrer et pour signer.

3.4.1 IMPLEMENTATION : cahier des charges de la phase 4

Implémenter les fonctionnalités suivantes et les ajouter à l'interprète de la phase précédente :

1. générer des clés de signature
2. générer le digest d'un fichier (chiffré ou clair)
3. chiffrer le digest avec la clef de chiffrement
4. signer le digest chiffré avec la clef privée de signature
5. vérifier une signature avec la clé publique du signataire
6. vérifier et déchiffrer le message (digest) signé et chiffré
7. ajouter la gestion des clefs de signatures à l'annuaire précédent.

3.5 PHASE 5 : Registre d'authentification des clefs publiques

Il existe des registres de clefs publiques dont l'authenticité est garantie (les clefs et l'identité du propriétaire) par des autorités de certifications. Ces certifications peuvent elles-mêmes être certifiées par d'autres autorités. La certification est un problème délicat dans la mesure où certaines autorités sont connues pour vendre des certificats de complaisance.

Les autorités de certifications sont des organismes publiques ou privés selon un modèle centralisé. Nous allons simuler une telle autorité en utilisant un registre, implémenté par une blockchain comparable à celle qui gère le Bitcoin (entre autre).

Elle va être très simplifiée

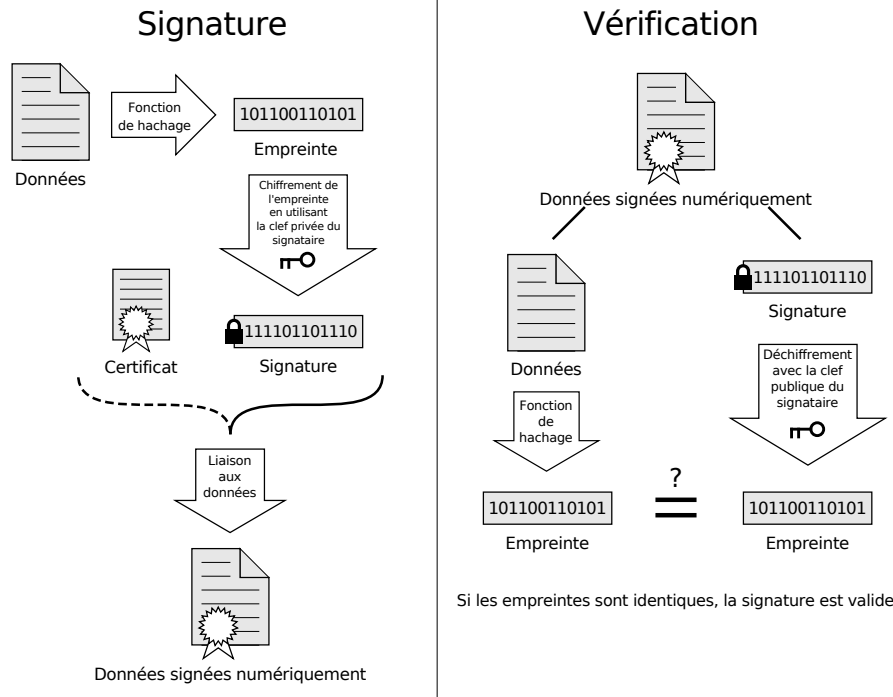


FIGURE 1 – Signature d'un documents par Guilieb Travail personnel,
CC BY-SA 3.0, <https://commons.wikimedia.org/w/index.php?curid=45669258>

3.5.1 Cahier des charges d'implémentation

Le header bc.h décrit la structure de la blockchain, des blocs et des transactions.

Entités gérées :

- Institution : l'institution qui propose le service de certification (CIC, gafam, ministère, tec.)
- Clients = $user_1, \dots, user_n$: personnes physiques ou morales voulant faire certifier leurs clefs.

Principe général de la boucle principale

1. Un menu permet de créer une nouvelle transaction d'un utilisateur vers l'Institution. La transaction correspond à un évènement : déclaration ou révocation d'une clef. L'opération est facturée N tokens à l'utilisateur. Cette somme est débité sur le compte de l'utilisateur (on vérifie qu'il a les moyens).
2. La transaction est mise en file
3. Un mineur est tiré au hasard parmi les utilisateurs. Il va chercher des transactions dans la file, les vérifie, les mine, crée un nouveau bloc qui est ajouté à la blockchain et reçoit n tokens ($n < N$).

Résumé du travail :

1. Structure de données pour la blockchain, les blocs et les transactions
2. Code pour initialiser la blockchain (création du génésis non miné et sans transaction)
3. file pour gérer les nouvelles transactions
4. Calcul du hash256 des blocs, arbres de Merkle et les transactions
5. Minage des blocs. Le mineur reçoit de l'Institution une rétribution en tokens.
6. Test d'intégrité (présence du Génésis, les ash des blocks sont cohérents, le chaînage aussi, les racines de Merkle aussi)
7. Génération d'un blocks à partir des nouvelles transactions

8. Implémenter des comptes utilisateurs initialisés à 100 tokens, et un compte pour l'Institution initialisé à 1000 tokens.
9. Faire un main qui gère la boucle principale

3.5.2 Fonctionnalités supplémentaires

- affichage de la blockchain ou d'un bloc d'un numéro donné
- recherche d'une clé pour trouver le propriétaire
- recherche de toutes les clés de quelqu'un
- savoir si une clé est révoquée
- date de création
- événements :
 - diffusion d'une clef cryptographique publique, code : NCK (new cryptographic key) suivi d'une clef
 - révocation d'une clef cryptographique publique, code : RCK (revoke cryptographic key) suivi d'une clef
 - création d'une clef de vérification de signature, code : NSK (new sign key) suivi d'une clef
 - révocation d'une clef de vérification de signature, code : RSK (revoke sign key) suivi d'une clef
- identité de l'émetteur (email et clefs publiques)